INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Listas filtrables con LazyColumn en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	08	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	B
FECHA DE PRESENTACIÓN:	24/10/2026	**HORA DE PRESENTACIÓN:**	11:59:00		
INTEGRANTE(s): - Hancco Mullisaca, Sergio Danilo - Huacani Jara, Denise Andrea				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Link Repositorio 1: https://github.com/shanccom/IDNP_Repo/tree/main/lab08_ListaFiltrable_Compose

Link Repositorio 2:https://github.com/DeniseHuacani/IDNP-LABORATORIO/tree/main/lab08/ListaFiltrable_Compose

Ejercicio Resuelto por el Docente

A continuación se detallan, paso a paso, las instrucciones para mostrar una lista de edificios del campus con un campo de búsqueda que filtra en tiempo real. También puede apoyarse en el siguiente video tutorial:

Video de referencia: «LazyColumn en Android Jetpack Compose. Lista con texto y botones» https://www.youtube.com/watch?v=C4EkfVFC0cg

Pasos para la Construcción de la Aplicación

1. **Paso 1. Crear un nuevo proyecto:**
   - **Nombre sugerido para el proyecto:** `ListaFiltrable_Compose`
   - **Package name:** `com.example.listafiltrable_compose`
   - **Language:** Kotlin
   - **Minimum SDK:** Valor por defecto de la plantilla (emulador o dispositivo físico con API 34 o superior).

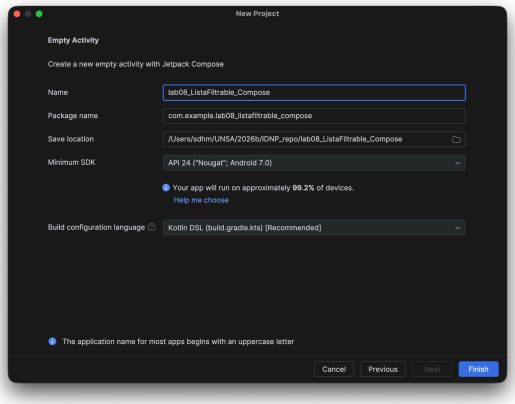


Figura 1: Paso 1: Creación del nuevo proyecto en Android Studio con plantilla Empty Activity (Compose).

2. **Paso 2. Agregar importaciones requeridas:**
   - Agregar, debajo de la declaración del `package` y antes de la clase `MainActivity`, todas las importaciones necesarias para el ciclo de vida, componentes de Jetpack Compose (`Column`, `Spacer`, `LazyColumn`, `items`, `Card`, `OutlinedTextField`, `Surface`, `Text`), estados reactivos (`remember`, `mutableStateOf`, `getValue`, `setValue`) y utilidades de estilo.

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Card
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```

Figura 2: Paso 2: Inclusión de paquetes e importaciones de Jetpack Compose en MainActivity.kt.

3. **Paso 3. Definir la data class del modelo de datos:**
   - Declarar fuera de cualquier Composable la data class `Edificio` encargada de estructurar los registros, definiendo los campos `id` (Int), `nombre` (String), `categoria` (String) y `descripcion` (String).

```
data class Edificio(
    val id: Int,
    val nombre: String,
    val categoria: String,
    val descripcion: String
)
```

Figura 3: Paso 3: Definición de la data class Edificio para modelar la información.

4. **Paso 4. Crear el Composable `ListaEdificios` con estado base:**
   - Declarar la función Composable `ListaEdificios`.
   - Crear una colección inmutable de al menos 6 edificios de muestra persistida en memoria mediante `remember { listOf(...) }`.
   - Definir una variable de estado mutable para rastrear el texto de búsqueda (`texto`) inicializada en cadena vacía utilizando `remember { mutableStateOf("") }`.
5. **Paso 5. Incorporar el campo de búsqueda (`OutlinedTextField`):**
   - Estructurar el diseño vertical dentro de un contenedor `Column` ocupando el tamaño completo con espaciado interno.
   - Insertar un campo de texto `OutlinedTextField` vinculado bidireccionalmente a la variable `texto`, actualizándola mediante el callback `onValueChange` y asignándole la etiqueta «Buscar edificio».
   - Añadir un separador `Spacer` vertical inferior para distanciar el campo de la lista.
6. **Paso 6. Calcular la lista filtrada:**
   - Procesar la lista original a partir del contenido de `texto` utilizando el operador de filtrado `.filter { ... }`.
   - Comprobar que el nombre del edificio contenga el texto buscado ignorando mayúsculas y minúsculas (`ignoreCase = true`), permitiendo recalcular la lista filtrada de manera reactiva ante cada recomposición.
7. **Paso 7. Declarar el contenedor `LazyColumn`:**
   - Implementar el componente `LazyColumn` a pantalla completa.
   - Invocar la función constructora `items()` pasando la lista de edificios filtrados y proveyendo una clave única estable (`key = { it.id }`) para optimizar el reciclaje de vistas.
   - Delegar la renderización de cada elemento al Composable de fila `FilaEdificio`.

```
@Composable
fun ListaEdificios() {
    val edificios = remember {
        listOf(
            Edificio( id = 1,  nombre = "Biblioteca Central",  categoria = "Servicios",  descripcion = "Présta
            Edificio( id = 2,  nombre = "Comedor Universitario",  categoria = "Servicios",  descripcion = "Ali
            Edificio( id = 3,  nombre = "Facultad de Ingeniería",  categoria = "Académico",  descripcion = "Au
            Edificio( id = 4,  nombre = "Facultad de Ciencias",  categoria = "Académico",  descripcion = "Aula
            Edificio( id = 5,  nombre = "Coliseo Universitario",  categoria = "Deportes",  descripcion = "Acti
            Edificio( id = 6,  nombre = "Auditorio Central",  categoria = "Eventos",  descripcion = "Conferenc
        )
    }
    var texto by remember { mutableStateOf( value = "") }
    Column(modifier = Modifier.fillMaxSize().padding( all = 16.dp)) {
        OutlinedTextField(
            value = texto,
            onValueChange = { texto = it },
            label = { Text("Buscar edificio") },
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(modifier = Modifier.height(12.dp))
        val edificiosFiltrados = edificios.filter {
            it.nombre.contains( other = texto, ignoreCase = true)
        }

        LazyColumn(modifier = Modifier.fillMaxSize()) {
            items(edificiosFiltrados, key = { it.id }) { edificio ->
                FilaEdificio(edificio)
            }
        }
    }
}
```

Figura 4: Pasos 4 al 7: Construcción integrada de ListaEdificios con filtrado reactivo y LazyColumn.

8. **Paso 8. Implementar el Composable `FilaEdificio`:**
   - Definir la función Composable `FilaEdificio` que recibe un objeto `Edificio`.
   - Renderizar una tarjeta visual `Card` con márgenes verticales y espaciado interno.
   - Organizar en una disposición vertical (`Column`) al menos tres datos representativos: el nombre (destacado en negrita y tamaño amplio), la categoría (con tamaño reducido y color secundario) y la descripción general.

```
1 Usage
@Composable
fun FilaEdificio(edificio: Edificio) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 6.dp)
    ) {
        Column(modifier = Modifier.padding( all = 12.dp)) {
            Text(
                text = edificio.nombre,
                fontWeight = FontWeight.Bold,
                fontSize = 18.sp
            )
            Text(
                text = edificio.categoria,
                fontSize = 14.sp,
                color = Color.Gray
            )
            Text(
                text = edificio.descripcion,
                fontSize = 14.sp
            )
        }
    }
}
```

Figura 5: Paso 8: Implementación del Composable FilaEdificio utilizando un contenedor Card.

9. **Paso 9. Configurar la ejecución en `MainActivity` y la vista previa:**

- Configurar dentro del método `onCreate` de `MainActivity` el bloque `setContent`, envolviendo el Composable principal `ListaEdificios` dentro del tema del sistema (`ListaFiltrable_ComposeTheme`) y un contenedor `Surface`.
- Añadir un Composable de previsualización anotado con `@Preview` (`ListaEdificiosPreview`) para validar visualmente la interfaz desde el entorno de diseño de Android Studio.

```
1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Lab08_ListaFiltrable_ComposeTheme {
                Surface(modifier = Modifier.fillMaxSize()) {
                    ListaEdificios()
                }
            }
        }
    }
}
@Preview(showBackground = true)
@Composable
fun ListaEdificiosPreview() {
    Lab08_ListaFiltrable_ComposeTheme {
        ListaEdificios()
    }
}
```

Figura 6: Paso 9: Punto de entrada en MainActivity.kt y función Composable con `@Preview`.

10. **Paso 10. Ejecución y prueba interactiva en el emulador:**
    - Ejecutar la aplicación en el emulador o dispositivo de prueba con API 34+.
    - Escribir distintas cadenas de caracteres en el campo de búsqueda y constatar que la lista se recompone en tiempo real mostrando exclusivamente las coincidencias por cada pulsación.

Estudiante 1



Figura 7: EStudiante 1 : Lista inicial con todos los edificios.

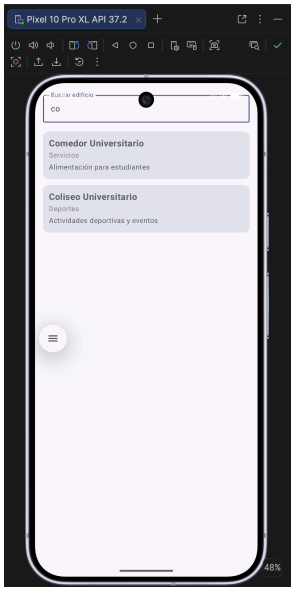


Figura 8: Estudiante 1 : Filtrado interactivo en tiempo real.

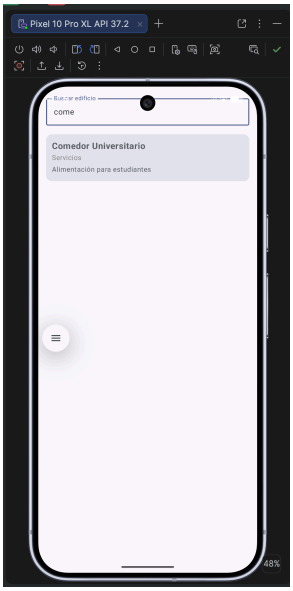


Figura 9: Estudiante 1 : Coincidencia refinada específica.

Estudiante 2



Figura 10: Estudiante 2 : Lista inicial con todos los edificios.

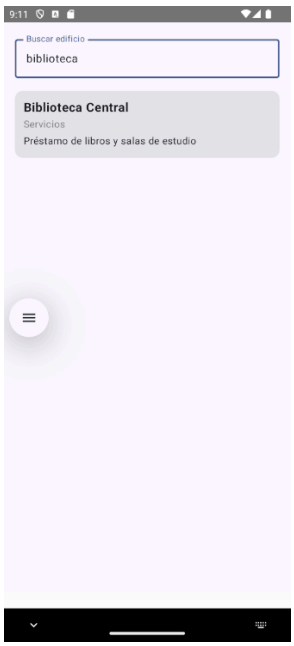


Figura 11: Estudiante 2 : Busqueda de biblioteca

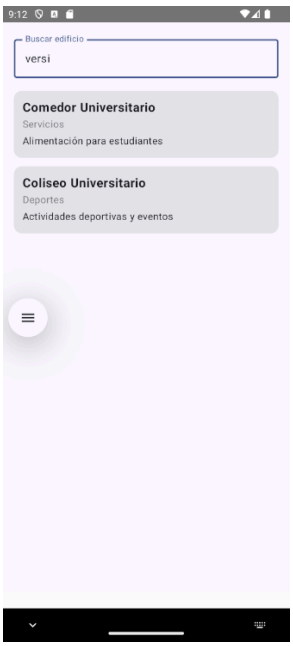


Figura 12: Estudiante 2 : Busqueda que contengan la palabra «versi»

1. **Paso 11. Validación de escenarios límite:**
   - Ingresar intencionalmente términos o palabras clave que no coincidan con ningún registro.
   - Verificar que la lista queda vacía de forma limpia y transparente, sin generar cuelgues ni errores en la aplicación.

Estudiante 1

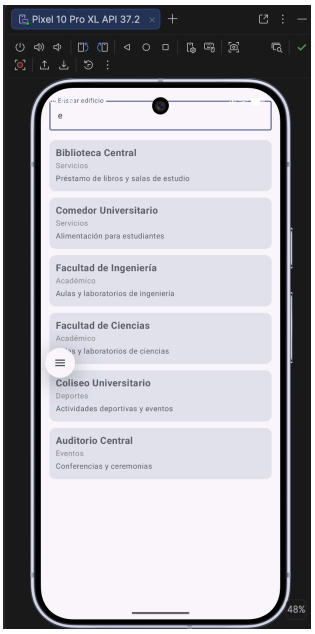


Figura 13: Paso 11 (a): Búsqueda sin coincidencias registradas.

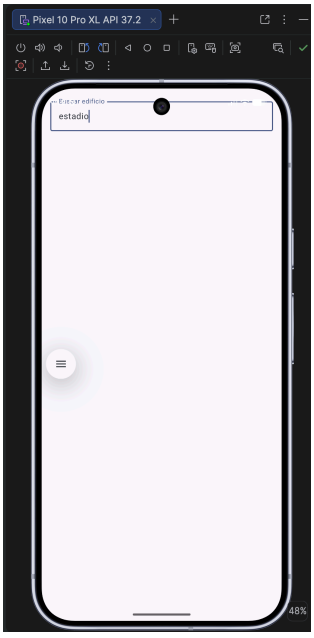


Figura 14: Paso 11 (b): Lista vacía sin errores en el emulador.

Estudiante 2

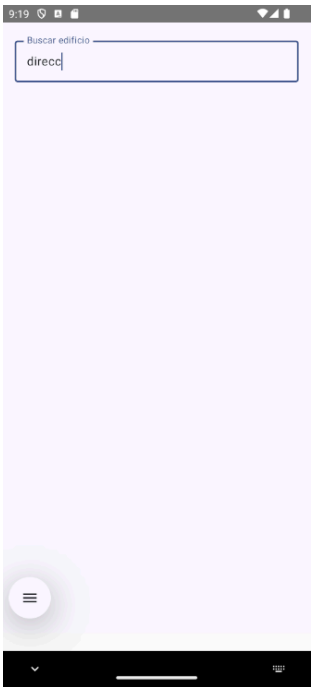


Figura 15: Estudiante 2 – Paso 11 (a): Búsqueda sin coincidencias registradas.

Ejercicio Propuesto: Ampliación de la Lista de Edificaciones Históricas del Campus

Enunciado

Amplía el ejercicio resuelto para mostrar la lista completa de edificaciones históricas del campus:

1. **Persistencia:** Reemplaza la lista de muestra por al menos 8 edificios reales del campus, cargados desde un archivo de texto plano (similar al patrón del Laboratorio 03) en lugar de estar escritos directamente en el código.
2. **Identidad visual:** Agrega a `FilaEdificio` un ícono o imagen representativa (puede ser un `Icon` de Material Icons).
3. **Filtro avanzado:** Permite elegir el criterio de filtro agregando un segundo campo o menú desplegable para filtrar por categoría además de por nombre.
4. **(Reto opcional) Integración:** Integra esta lista filtrable como reemplazo de `EdificiosScreen` (Laboratorio 05/06), combinándola con las tarjetas `TarjetaLugar` o `TarjetaCompacta` del Laboratorio 06 dentro de cada fila.

Desarrollo Paso a Paso

- Propuesta 1

Paso 1. Carga de Edificaciones Reales desde Archivo de Texto Plano (Persistencia)

Para desacoplar los datos de la interfaz gráfica y asegurar persistencia configurable, se sustituyó la lista estática en memoria por una lectura dinámica desde almacenamiento interno, replicando el patrón implementado en el Laboratorio 03. Se creó el archivo `app/src/main/assets/edificios.txt` con 12 edificaciones reales del campus de la UNSA (Rectorado, Biblioteca Central, Aula Magna, facultades, comedor universitario, coliseo, complejo deportivo y centro de cómputo), organizadas por líneas en formato `id,nombre,categoria,descripcion`, omitiendo automáticamente comentarios con `#` y líneas en blanco.

```
# Formato: id,nombre,categoria,descripcion
1,Rectorado UNSA,Administrativo,Sede administrativa y despacho de las autoridades universitarias
```

```
2,Biblioteca Central UNSA,Servicios,Préstamo de libros y salas de estudio para estudiantes
3,Aula Magna UNSA,Eventos,Conferencias y ceremonias académicas de la universidad
4,Facultad de Ingeniería de Sistemas,Académico,Aulas y laboratorios de computación e informática
5,Facultad de Ingeniería de Producción y Servicios,Académico,Aulas y laboratorios de ingeniería
6,Facultad de Medicina UNSA,Académico,Aulas y laboratorios de ciencias de la salud
7,Facultad de Ciencias Económicas,Académico,Aulas y oficinas de las escuelas de economía
8,Comedor Universitario UNSA,Servicios,Alimentación para estudiantes y personal universitario
9,Coliseo Universitario UNSA,Deportes,Actividades deportivas y eventos masivos
10,Complejo Deportivo UNSA,Deportes,Canchas y espacios para la práctica del deporte
11,Centro de Cómputo UNSA,Servicios,Salas de cómputo y soporte informático
12,Facultad de Arquitectura y Urbanismo,Académico,Talleres de diseño y aulas de arquitectura
```

En `MainActivity.kt` se eliminó la lista cableada y se incorporó la función `parsearEdificios(contenido)`, encargada de transformar el texto plano en una colección `List<Edificio>`. La función `cargarEdificios(context)` gestiona el ciclo de vida del archivo copiándolo desde `assets` hacia el almacenamiento interno (`filesDir`) durante el primer arranque y consumiéndolo posteriormente mediante `openFileInput("edificios.txt").bufferedReader()`. Finalmente, el composable `ListaEdificios` recupera los datos mediante `LocalContext.current` y delega la renderización a `ListaEdificiosContenido(edificios)`.

```kotlin
fun parsearEdificios(contenido: String): List<Edificio> = contenido.lineSequence()
    .map { it.trim() }
    .filter { it.isNotEmpty() && !it.startsWith("#") }
    .mapNotNull { linea ->
        val partes = linea.split(",")
        if (partes.size >= 4) {
            Edificio(partes[0].trim().toIntOrNull() ?: 0, partes[1].trim(), partes[2].trim(),
partes.drop(3).joinToString(",").trim())
        } else null
    }.toList()

fun cargarEdificios(context: Context): List<Edificio> {
    val archivo = File(context.filesDir, "edificios.txt")
    if (!archivo.exists()) {
        context.assets.open("edificios.txt").use { entrada ->
            archivo.outputStream().use { salida -> entrada.copyTo(salida) }
        }
    }
    return parsearEdificios(context.openFileInput("edificios.txt").bufferedReader().use
{ it.readText() })
}
```

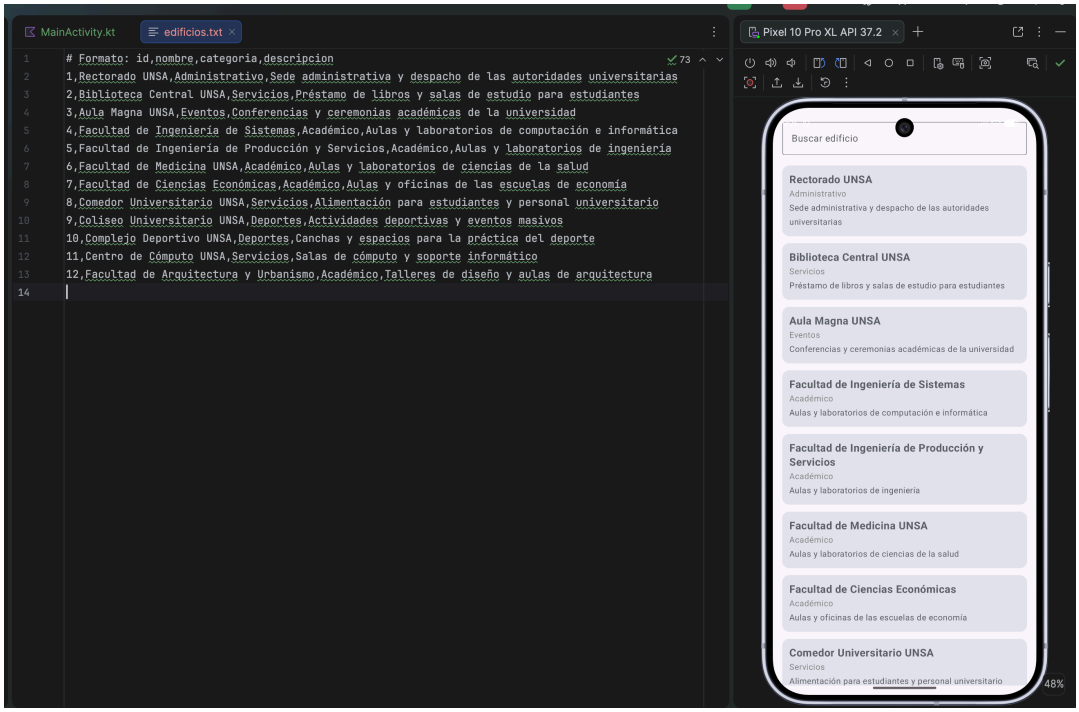


Figura 16: Paso 1: Archivo edificios.txt empaquetado en assets y lectura de los 12 registros en el emulador.

Paso 2. Incorporación de Ícono Representativo en `FilaEdificio`

Con el propósito de reforzar la jerarquía visual y facilitar la rápida identificación de las edificaciones, se incorporaron íconos vectoriales temáticos de `Material Icons`. Se estructuró la clase auxiliar `EstiloCategoria` con las propiedades `icono`, `colorFondo` y `colorIcono`, junto con la función `estiloCategoria(categoria)` que evalúa el texto y asigna dinámicamente vectores alusivos: `School` para Académico, `Restaurant` para Servicios, `SportsSoccer` para Deportes, `Event` para Eventos, `Business` para Administrativo y `Place` por defecto.

A nivel de interfaz, `FilaEdificio` se rediseñó sobre una disposición horizontal (`Row`) dentro de un contenedor `Card`. En la sección izquierda se despliega una insignia circular de `48.dp` (`clip(CircleShape)`) con el fondo tonal y el ícono vectorial correspondiente, mientras que a la derecha se organizan en columna el nombre del edificio en negrita, la categoría y la descripción detallada, asignando `Modifier.weight(1f)` para aprovechar fluidamente todo el ancho disponible.

```
private data class EstiloCategoria(val icono: ImageVector, val colorFondo: Color, val colorIcono:
Color)

private fun estiloCategoria(categoria: String): EstiloCategoria = when (categoria.lowercase()) {
    "académico", "academico" -> EstiloCategoria(Icons.Default.School, Color(0xFFE3F2FD),
Color(0xFF1565C0))
    "servicios" -> EstiloCategoria(Icons.Default.Restaurant, Color(0xFFFFF3E0), Color(0xFFEF6C00))
    "deportes" -> EstiloCategoria(Icons.Default.SportsSoccer, Color(0xFFE8F5E9),
Color(0xFF2E7D32))
    "eventos" -> EstiloCategoria(Icons.Default.Event, Color(0xFFF3E5F5), Color(0xFF6A1B9A))
    "administrativo" -> EstiloCategoria(Icons.Default.Business, Color(0xFFECEFF1),
Color(0xFF37474F))
    else -> EstiloCategoria(Icons.Default.Place, Color(0xFFF5F5F5), Color.Gray)
}

```

```
@Composable
fun FilaEdificio(edificio: Edificio) {
    val estilo = estiloCategoria(edificio.categoria)
    Card(
        modifier = Modifier.fillMaxWidth().padding(vertical = 4.dp),
        colors = CardDefaults.cardColors(containerColor = Color(0xFFF0F2F5))
    ) {
        Row(modifier = Modifier.fillMaxWidth().padding(12.dp), verticalAlignment =
Alignment.CenterVertically) {
            Box(
                modifier = Modifier.size(48.dp).clip(CircleShape).background(estilo.colorFondo),
                contentAlignment = Alignment.Center
            ) {
                Icon(estilo.icono, contentDescription = edificio.categoria, tint =
estilo.colorIcono, modifier = Modifier.size(24.dp))
            }
            Spacer(modifier = Modifier.width(12.dp))
            Column(modifier = Modifier.weight(1f)) {
                Text(edificio.nombre, fontWeight = FontWeight.Bold, fontSize = 16.sp, color =
Color.Black)
                Text(edificio.categoria, fontSize = 13.sp, fontWeight = FontWeight.SemiBold, color
= estilo.colorIcono)
                Spacer(modifier = Modifier.height(2.dp))
                Text(edificio.descripcion, fontSize = 12.sp, color = Color.DarkGray)
            }
        }
    }
}
```

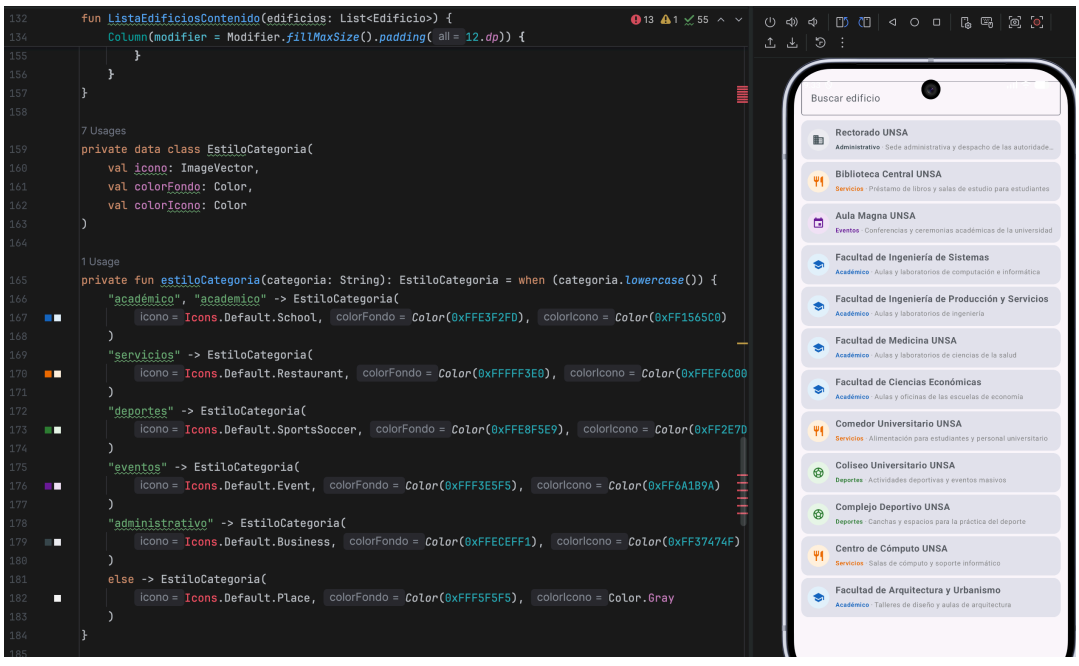


Figura 17: Paso 2: Insignias circulares temáticas de Material Icons integradas en FilaEdificio.

Paso 3. Criterio de Filtro Dual (por Nombre y Menú Desplegable por Categoría)

El sistema de búsqueda se expandió mediante un filtrado reactivo simultáneo que combina la entrada de texto con la selección de categorías mediante un menú desplegable `ExposedDropdownMenuBox`. Dentro de `ListaEdificiosContenido` se definen dos estados mutables: `texto` para el término buscado y `categoriaSeleccionada`, inicializado con la constante `TODAS_LAS_CATEGORIAS = "Todas"`. Las opciones del menú se obtienen dinámicamente desde los registros mediante `remember(edificios)`, asegurando que cualquier categoría añadida a futuro en `edificios.txt` se refleje de inmediato sin modificar el código fuente.

La lógica de filtrado se centralizó en la función `filtrarEdificios`, evaluando la conjunción lógica (`&&`) de dos condiciones: `coincideNombre` valida que el nombre contenga la subcadena ingresada (o admite todos si el campo está vacío) y `coincideCategoria` verifica coincidencia exacta o valida todas ante la opción `"Todas"`. En la interfaz, ambos componentes se ubican en una misma fila horizontal con `Arrangement.spacedBy(8.dp)`, incorporando mejoras de identidad visual como un fondo cálido institucional (`#F7F4EE`), tarjetas blancas con esquinas de 14dp, acento verde en campos enfocados y scroll optimizado en `LazyColumn`.

```
1  private const val TODAS_LAS_CATEGORIAS = "Todas"
2
3  fun filtrarEdificios(edificios: List<Edificio>, nombre: String, categoria: String): List<Edificio>
{
4      val buscado = nombre.trim()
5      return edificios.filter { ed ->
6          val coincideNombre = buscado.isEmpty() || ed.nombre.contains(buscado, ignoreCase = true)
7          val coincideCategoria = (categoria == TODAS_LAS_CATEGORIAS) ||
ed.categoria.equals(categoria, ignoreCase = true)
8          coincideNombre && coincideCategoria
9      }
10 }
11
12 // En ListaEdificiosContenido:
13 var texto by remember { mutableStateOf("") }
14 var categoriaSeleccionada by remember { mutableStateOf(TODAS_LAS_CATEGORIAS) }
15 val categorias = remember(edificios) { listOf(TODAS_LAS_CATEGORIAS) + edificios.map
{ it.categoria }.distinct().sorted() }
16 val listaFiltrada = remember(texto, categoriaSeleccionada, edificios)
{ filtrarEdificios(edificios, texto, categoriaSeleccionada) }
17
18 Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
19     OutlinedTextField(value = texto, onValueChange = { texto = it }, label = { Text("Nombre") },
modifier = Modifier.weight(1f))
20     FiltroCategoria(categorias = categorias, seleccionada = categoriaSeleccionada, onSeleccionar =
{ categoriaSeleccionada = it }, modifier = Modifier.weight(1f))
21 }
```



Figura 18: Filtro inicial «Todas» sobre fondo cálido `\#F7F4EE` y tarjetas de 14dp.



Figura 19: Filtrado por categoría «Académico» acotado a las facultades.

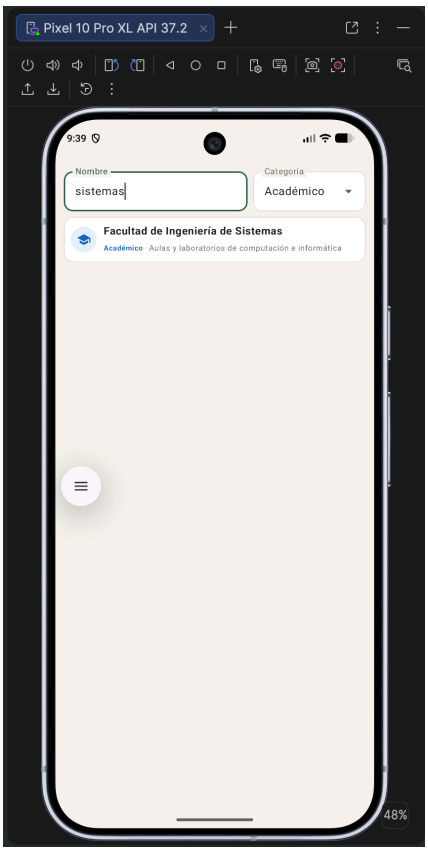


Figura 20: Filtro dual interactivo: búsqueda «sistemas» dentro de «Académico».

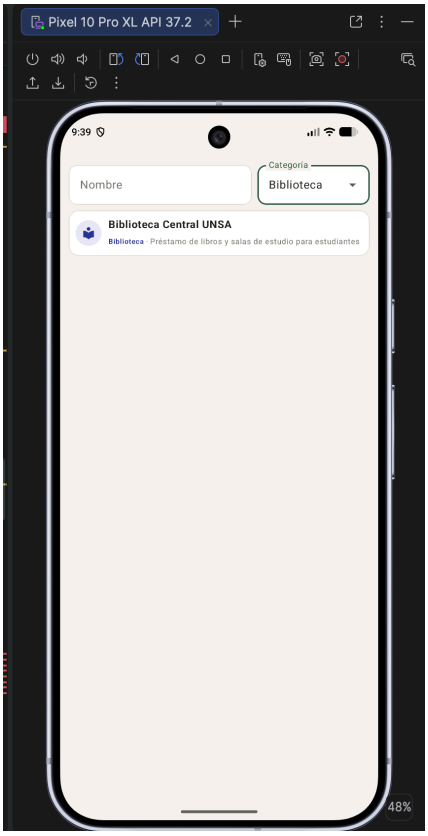


Figura 21: Filtrado dinámico específico para la categoría «Biblioteca».

Paso 4. (Reto Opcional) Integración con el Patrón de TarjetaLugar del Laboratorio 06

Para resolver el reto opcional, se integró el lenguaje visual desarrollado en el Laboratorio 06 sustituyendo el diseño base de fila por una versión adaptada de `TarjetaLugar`. La tarjeta se implementa sobre un `Surface` con esquinas redondeadas de 16dp, elevación controlada y un contenedor frontal de `52.dp` que resalta el ícono representativo de la categoría. A su lado, una columna organiza el nombre del edificio, su descripción y una etiqueta inferior en forma de chip que expone la categoría temática en lugar del dato de distancia del laboratorio anterior.

Esta pantalla adaptada preserva de forma transparente todo el motor de filtrado por texto y categoría, permitiendo que `EdificiosScreenIntegrada` actúe como un reemplazo directo y enriquecido de las pantallas de navegación previas sin perder el rendimiento del reciclaje de vistas provisto por `LazyColumn`.

```
@Composable
fun TarjetaLugar(edificio: Edificio, onClick: () -> Unit = {}) {
    val estilo = estiloCategoria(edificio.categoria)
    Surface(
        modifier = Modifier.fillMaxWidth().clickable(onClick = onClick),
        shape = RoundedCornerShape(16.dp),
        color = Color.White,
        border = BorderStroke(1.dp, Color(0xFFE2E8F0)),
        shadowElevation = 1.dp
    ) {
```

```
        Row(modifier = Modifier.padding(10.dp), verticalAlignment = Alignment.CenterVertically) {
            Box(
                modifier =
Modifier.size(52.dp).clip(RoundedCornerShape(14.dp)).background(estilo.colorFondo),
                contentAlignment = Alignment.Center
            ) {
                Icon(estilo.icono, contentDescription = edificio.categoria, tint =
estilo.colorIcono, modifier = Modifier.size(28.dp))
            }
            Spacer(modifier = Modifier.width(12.dp))
            Column(modifier = Modifier.weight(1f)) {
                Text(edificio.nombre, fontWeight = FontWeight.Bold, fontSize = 15.sp, color =
Color.Black)
                Text(edificio.descripcion, fontSize = 12.sp, color = Color.DarkGray, maxLines = 2)
                Spacer(modifier = Modifier.height(4.dp))
                Surface(shape = RoundedCornerShape(6.dp), color = estilo.colorIcono.copy(alpha =
0.12f)) {
                    Text(edificio.categoria, fontSize = 11.sp, fontWeight = FontWeight.SemiBold,
color = estilo.colorIcono, modifier = Modifier.padding(horizontal = 8.dp, vertical = 2.dp))
                }
            }
        }
    }
}
```

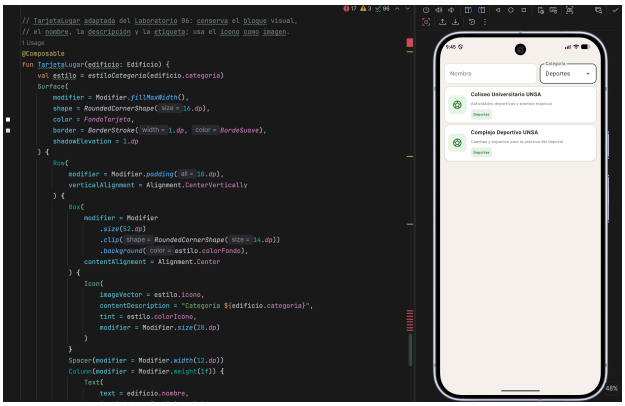


Figura 22: Integración de TarjetaLugar con insignias de categoría.

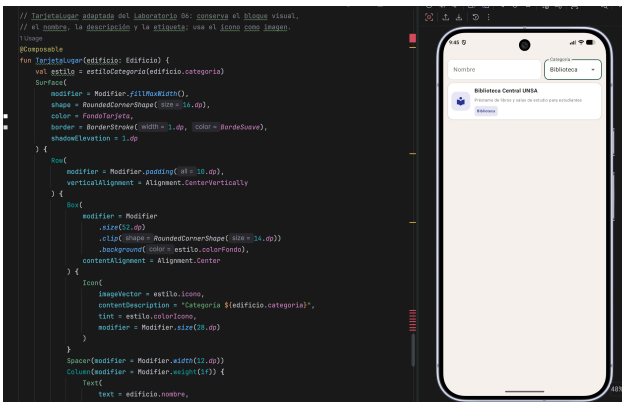


Figura 23: Filtrado combinado interactivo sobre TarjetaLugar.

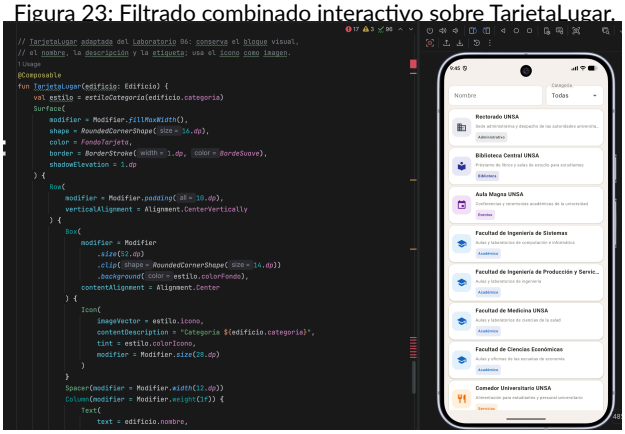


Figura 24: Desplazamiento fluido y consistencia gráfica en LazyColumn.

- Propuesta 2

Paso 1: Cargar los edificios desde un archivo de texto plano

Se creó la carpeta `assets` en el módulo `app` y dentro el archivo `edificios.txt`. Luego se escribió la función `cargarEdificios()`, que abre el archivo, lo lee línea por línea, descarta las líneas vacías o mal formadas y convierte cada línea en un objeto `Edificio`. En `ListaEdificios` la lista ya no se escribe a mano: se obtiene con `remember { cargarEdificios(context) }`, de modo que el archivo se lee una sola vez y no en cada recomposición.

Archivo edificios.txt:

```
1|Biblioteca Central|Servicios|Préstamo de libros y salas de estudio
2|Comedor Universitario|Servicios|Alimentación para estudiantes
3|Facultad de Ingeniería de Producción y Servicios|Académico|Aulas y laboratorios de ingeniería
4|Facultad de Ingeniería de Procesos|Académico|Laboratorios de química y procesos
5|Facultad de Ciencias Naturales y Formales|Académico|Aulas y laboratorios de ciencias
6|Coliseo Universitario|Deportes|Actividades deportivas y eventos
7|Auditorio Central|Eventos|Conferencias y ceremonias
8|Rectorado|Administrativo|Oficinas de gobierno universitario
9|Facultad de Derecho|Académico|Aulas y salas de práctica jurídica
```

Código implementado:

```kotlin
// Nuevo import
import android.content.Context
import androidx.compose.ui.platform.LocalContext

// Lee assets/edificios.txt (formato: id|nombre|categoria|descripcion)
fun cargarEdificios(context: Context): List<Edificio> =
    context.assets.open("edificios.txt").bufferedReader().useLines { lineas ->
        lineas
            .filter { it.isNotBlank() }
            .map { it.split("|") }
            .filter { it.size == 4 }
            .mapNotNull { p ->
                p[0].trim().toIntOrNull()?.let { id ->
                    Edificio(id, p[1].trim(), p[2].trim(), p[3].trim())
                }
            }
            .toList()
    }

// En ListaEdificios: se reemplaza la lista escrita en el código
val context = LocalContext.current
val edificios = remember { cargarEdificios(context) }
```

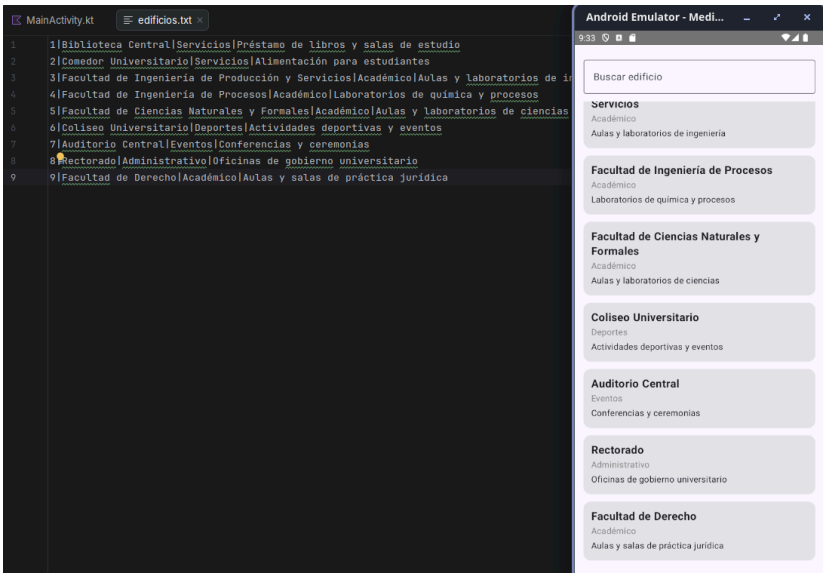


Figura 25: Paso 1 (b): lista completa cargada desde el archivo.

,

Paso 2: Agregar un ícono a `FilaEdificio`

Se agregó la dependencia `material-icons-extended` para disponer de más íconos y se creó la función `iconoPorCategoria()`, que devuelve un ImageVector distinto según la categoría. En FilaEdificio la Column de textos pasó a estar dentro de un Row: a la izquierda se dibuja el ícono sobre un círculo de color y a la derecha los tres textos de siempre.

Código agregado (en el archivo build.gradle.kts, luego Sync Now):

```
implementation("androidx.compose.material:material-icons-extended")
```

Código agregado :

```
// Función nueva: ícono según la categoría
fun iconoPorCategoria(categoria: String): ImageVector = when (categoria) {
    "Servicios" -> Icons.Default.LocalLibrary
    "Académico" -> Icons.Default.School
    "Deportes" -> Icons.Default.SportsSoccer
    "Eventos" -> Icons.Default.Event
    "Administrativo" -> Icons.Default.Business
    else -> Icons.Default.Place
}

// FilaEdificio: el ícono va a la izquierda, los textos a la derecha
Row(
    modifier = Modifier.padding(12.dp),
    verticalAlignment = Alignment.CenterVertically
) {
    Box(
        modifier = Modifier
            .size(48.dp)
            .clip(CircleShape)
            .background(MaterialTheme.colorScheme.primaryContainer),
        contentAlignment = Alignment.Center
    ) {
        Icon(
            imageVector = iconoPorCategoria(edificio.categoria),
            contentDescription = edificio.categoria,
            tint = MaterialTheme.colorScheme.primary
        )
    }
    Spacer(modifier = Modifier.width(12.dp))
    Column { /* nombre, categoría y descripción, igual que antes */ }
}
```



Figura 26: Paso 2 (a): cada edificio muestra el ícono de su categoría.

,

Paso 3: Filtrar también por categoría con un menú desplegable

Se agregó un segundo criterio de filtro. La lista de categorías no se escribe a mano: se obtiene del propio archivo con `map { it.categoria }.distinct()` y se le antepone la opción «Todas». Se añadieron dos estados nuevos: `categoria` (la elegida) y `menuAbierto` (si el menú está desplegado).

Código agregado (`MainActivity.kt`):

```
// En ListaEdificios: estados nuevos
val categorias = remember(edificios) {
    listOf("Todas") + edificios.map { it.categoria }.distinct()
}
var categoria by remember { mutableStateOf("Todas") }
var menuAbierto by remember { mutableStateOf(false) }

// Menú desplegable, debajo del campo de búsqueda
Box {
    OutlinedButton(
        onClick = { menuAbierto = true },
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("Categoría: $categoria")
```

```
    }
    DropdownMenu(
        expanded = menuAbierto,
        onDismissRequest = { menuAbierto = false }
    ) {
        categorias.forEach { c ->
            DropdownMenuItem(
                text = { Text(c) },
                onClick = {
                    categoria = c
                    menuAbierto = false
                }
            )
        }
    }
}

// Filtro combinado: nombre Y categoría
val edificiosFiltrados = edificios.filter {
    it.nombre.contains(texto, ignoreCase = true) &&
        (categoria == "Todas" || it.categoria == categoria)
}
```



Figura 27: Paso 3 (a): menú desplegable con las categorías.

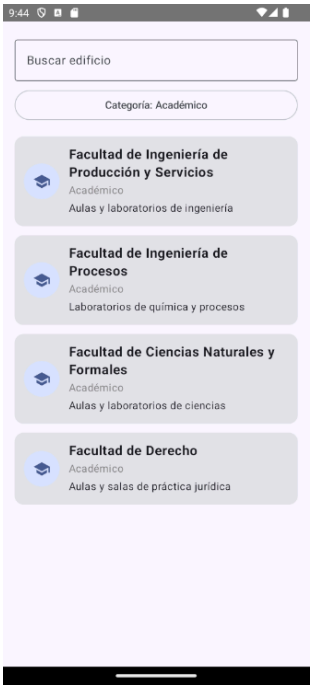


Figura 28: Paso 3 (b): lista filtrada por la categoría «Académico».

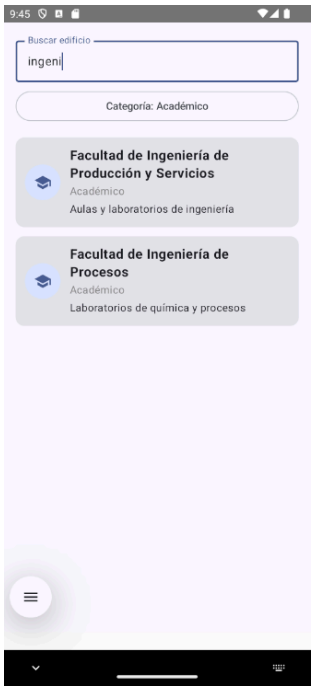


Figura 29: Paso 3 (c): filtro combinado: categoría «Académico» y texto «ingeniería».

Paso 4 (reto opcional): Integrar la lista como reemplazo de `EdificiosScreen`

La lista filtrable se integró en el proyecto del Laboratorio 05, que ya tiene la barra de navegación con cuatro pestañas y el `SeleccionViewModel` compartido. Para ello se copió `edificios.txt` a la carpeta `assets` de ese proyecto y se agregó la dependencia `material-icons-extended`. Los cambios fueron los siguientes:

- La data class `Edificio` pasó a tener cuatro campos (`id`, `nombre`, `categoria` y `descripcion`) y se añadieron `cargarEdificios()` e `iconoPorCategoria()`.
- `EdificiosScreen` ahora contiene el campo de búsqueda, el menú de categoría, un contador de resultados y la `LazyColumn` con una `key` estable.
- Cada fila usa `TarjetaCompacta`: ícono, nombre, categoría con descripción y el botón «Ver». Como se sigue usando el ViewModel, el botón llama a `viewModel.seleccionarEdificio()` y la tarjeta elegida se resalta. Como no se dispone del código del Laboratorio 06, esta tarjeta se definió en el mismo archivo.
- En `HomeScreen`, el total de edificios ya no es el número fijo «4», sino el tamaño de la lista leída del archivo.
- En `MapaScreen`, los pines se actualizaron con nombres que existen en el archivo.

El resto del código (`SeleccionViewModel`, navegación, Home, Perfil y tema) no se modificó.

Código agregado (`MainActivity.kt`):

```
// Data class ampliada (reemplaza a Edificio(nombre, descripcion))
data class Edificio(
    val id: Int,
    val nombre: String,
    val categoria: String,
    val descripcion: String
)

// cargarEdificios() e iconoPorCategoria(): iguales a los de los pasos 1 y 2

// HomeScreen: el total sale del archivo
val context = LocalContext.current
val totalEdificios = remember { cargarEdificios(context).size }
InfoCard(totalEdificios.toString(), "Edificios", Modifier.weight(1f))

// EdificiosScreen: lista filtrable (búsqueda + categoría) con ViewModel
@Composable
fun EdificiosScreen(viewModel: SeleccionViewModel) {
    val context = LocalContext.current
    val edificios = remember { cargarEdificios(context) }
    var texto by remember { mutableStateOf("") }
    var categoria by remember { mutableStateOf("Todas") }
    // ... campo de búsqueda y menú desplegable, igual que en el Paso 3

    val edificiosFiltrados = edificios.filter {
        it.nombre.contains(texto, ignoreCase = true) &&
            (categoria == "Todas" || it.categoria == categoria)
    }

    LazyColumn(modifier = Modifier.weight(1f)) {
        items(edificiosFiltrados, key = { it.id }) { edificio ->
            TarjetaCompacta(edificio = edificio, viewModel = viewModel)
        }
    }
}

```

```
// Cada fila: la selección se escribe directamente en el ViewModel
@Composable
fun TarjetaCompacta(edificio: Edificio, viewModel: SeleccionViewModel) {
    val seleccionado = viewModel.edificioSeleccionado == edificio.nombre
    // Card resaltada si está seleccionada: ícono + textos + botón
    Button(onClick = { viewModel.seleccionarEdificio(edificio.nombre) }) {
        Text("Ver")
    }
}

// MapaScreen: pines con nombres presentes en el archivo
PinMapa("Biblioteca Central", Modifier.align(BiasAlignment(-0.55f, -0.6f)))
PinMapa("Coliseo Universitario", Modifier.align(BiasAlignment(0.55f, -0.5f)))
PinMapa("Rectorado", Modifier.align(BiasAlignment(-0.55f, 0.55f)))
PinMapa("Auditorio Central", Modifier.align(BiasAlignment(0.6f, 0.6f)))
```



Figura 30: Paso 4 (a): pestaña Home con el total de edificios leído del archivo.

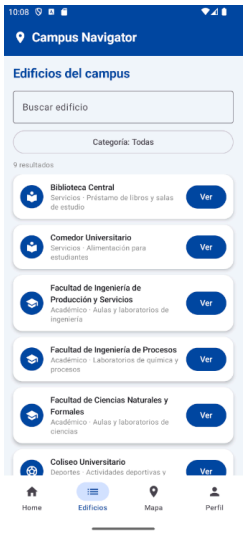


Figura 31: Paso 4 (b): pestaña Edificios con la lista filtrable y tarjetas compactas.

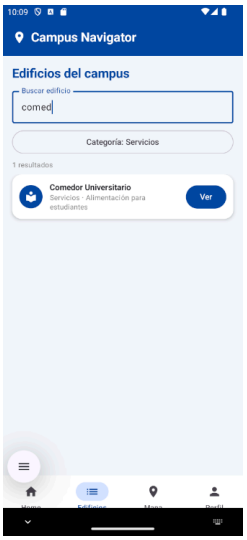


Figura 32: Paso 4 (c): filtro combinado por categoría y por nombre.

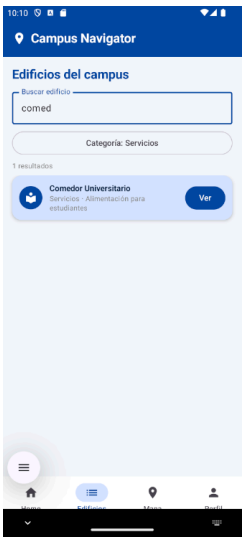


Figura 33: Paso 4 (d): tras pulsar «Ver», la tarjeta queda resaltada (ViewModel compartido).



Figura 34: Paso 4 (e): Home muestra el edificio elegido en «Último edificio consultado».

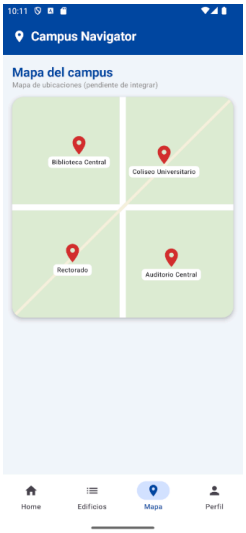


Figura 35: Paso 4 (f): pestaña Mapa con pines de edificios del archivo.

II. SOLUCIÓN DEL CUESTIONARIO

Pregunta 1: ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (lista de edificaciones que se filtra en tiempo real al escribir en el campo de búsqueda)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Sí, se resolvió de forma completa y satisfactoria el ejercicio propuesto por el docente (`ListaFiltrable_Compose`).

La solución implementó con éxito la arquitectura reactiva en Jetpack Compose:

1. **Estado de búsqueda reactivo:** Se vinculó el campo `OutlinedTextField` a una variable de estado mutable (`var texto by remember { mutableStateOf("") }`), notificando cada cambio de texto a través de su callback `onValueChange`.
2. **Filtrado en tiempo real:** Se procesó la lista de edificios con el operador `.filter { it.nombre.contains(texto, ignoreCase = true) }`, recalculando dinámicamente el subconjunto de datos coincidentes en cada recomposición sin retrasos de interfaz.
3. **Composición perezosa eficiente:** Se desplegó la lista con `LazyColumn`, asignando la clave única `key = { it.id }` y reutilizando el composable `FilaEdificio` envuelto en contenedores `Card`.
4. **Manejo de escenarios límite:** Se validó que al borrar el texto se restaure la lista íntegra, y al ingresar términos sin coincidencias la interfaz se mantenga estable y limpia sin provocar excepciones ni cierres forzados.

Evidencias de Ejecución en el Emulador
Estudiante 1



Figura 36: Lista inicial completa de edificios desplegada en el emulador.

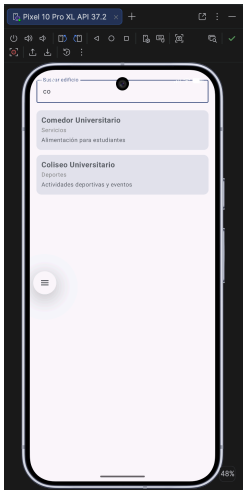


Figura 37: Filtrado interactivo en tiempo real al escribir en el buscador.

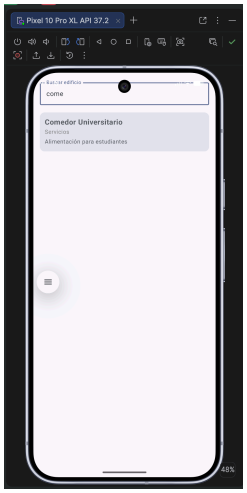


Figura 38: Coincidencia refinada mostrando la tarjeta específica.

Estudiante 2



Figura 39: Estudiante 2 : Lista inicial con todos los edificios.

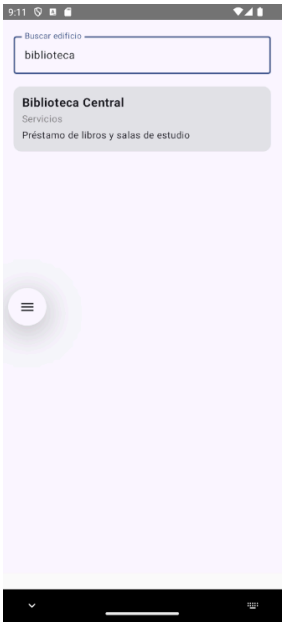


Figura 40: Estudiante 2 : Busqueda de biblioteca

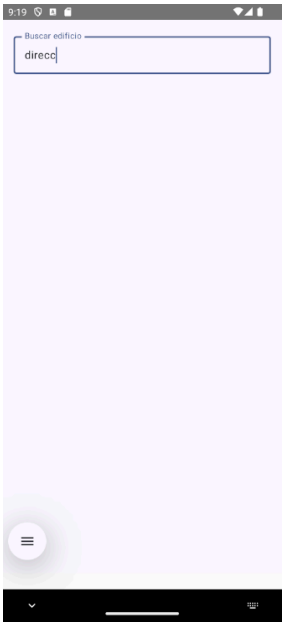


Figura 41: Estudiante 2 – Paso 11 (a): Búsqueda sin coincidencias registradas.

Pregunta 2: En el video de referencia se aborda el uso de LazyColumn. ¿Qué términos o conceptos nuevos, o que amplían lo visto en la sesión de teoría, se mencionan en el video? Explica brevemente.

El video de referencia aborda y profundiza conceptos esenciales para el diseño de interfaces escalables mediante `LazyColumn`:

1. **Sustitución declarativa de RecyclerView:** En contraste con el enfoque tradicional basado en vistas XML que exige clases intermedias (`RecyclerView.Adapter`, `ViewHolder`), inflado manual de layouts y métodos de enlace (`onBindViewHolder`),

`LazyColumn` simplifica la construcción de listas declarando únicamente el bloque de renderizado en un scope directo (`LazyListScope`).

2. **Diferenciación entre `item` e `items`:** Se detalla el uso de la función `item` (en singular) para renderizar elementos individuales o estáticos (como encabezados de sección, banners o separadores) intercalados de forma natural con bloques `items` (en plural) que procesan listas dinámicas.
3. **Uso de `itemsIndexed`:** Se explica la alternativa a `items` que proporciona tanto el índice numérico (`index`) como el elemento (`item`), facilitando casos de uso como numeración secuencial de filas, coloreado alterno o condicional según paridad, y manipulación directa por posición.

Pregunta 3: ¿Por qué es recomendable pasar un parámetro key = { it.id } a la función items() de una LazyColumn? ¿Qué podría ocurrir si no se especifica al filtrar o reordenar la lista?

Por defecto, cuando no se provee un parámetro `key`, `LazyColumn` utiliza la **posición numérica (índice 0, 1, 2...)** de cada elemento dentro de la lista para identificarlo en el árbol de composición.

Al especificar explícitamente `key = { it.id }` con un identificador único y estable:

- **Identidad semántica:** Jetpack Compose vincula la identidad persistente del objeto de datos con el nodo composable correspondiente en lugar de su ubicación transitoria.
- **Preservación de estado:** Permite al compilador y al runtime de Compose retener correctamente los estados internos (`remember`, variables mutables, animaciones, selecciones de usuario o foco) asociados a ese registro específico a lo largo de las recomposiciones.

Consecuencias de no especificar `key` al filtrar o reordenar

Si no se define una clave estable, ante cualquier modificación en la colección (filtrado por texto, selección de categoría, reordenamiento o eliminación) surgen fallas críticas:

1. **Pérdida o propagación errónea de estados locales (Bug de interfaz):** Si un ítem poseía un estado visual mutable (por ejemplo, una tarjeta expandida, un interruptor activado o un campo de texto), al cambiar el orden o filtrar elementos, dicho estado permanecerá anclado a la **posición ordinal**. En consecuencia, un elemento completamente diferente que ocupe esa posición heredará de manera indebida el estado previo.
2. **Degradación de rendimiento y parpadeos (Jank):** Compose no sabrá si un ítem se desplazó o si cambió por completo; por lo tanto, forzará la recomposición y redibujado innecesario de todos los composables visibles en lugar de simplemente reutilizar los nodos existentes.
3. **Fallo en animaciones de lista:** Los modificadores de animación (como `Modifier.animateItem()` o `animateItemPlacement()`) requieren obligatoriamente de una clave estable para calcular las trayectorias de interpolación espacial; sin ella, las transiciones de inserción, movimiento y descarte no pueden ejecutarse suavemente.

III. CONCLUSIONES

- Con la lista filtrable cambia como consecuencia del estado. Al declarar el texto de búsqueda con remember { mutableStateOf("") }, cada letra que el usuario escribe provoca una recomposición donde se vuelve a calcular la lista con .filter { ... }. Esto nos dejó claro que conviene tener una sola fuente de verdad para el estado y mantener el cálculo de la lista separado de la parte visual.
- También aprendimos a usar LazyColumn, que solo compone los elementos que se ven en pantalla, algo muy útil cuando hay muchos registros. Al usar items() con una clave estable (key = { it.id }) es por qué cada elemento necesita una identidad única y constante: sin ella se pierde el estado y se recomponen elementos sin necesidad, algo que no se nota con listas cortas en un Column normal.
- Al reemplazar la lista escrita directamente en Kotlin por la lectura de assets/edificios.txt, copiado al almacenamiento interno, es una ventaja para separar los datos de la presentación. ESto permite agregar nuevos edificios sin recompilar la app y reutilizar el mismo parser (parsearEdificios) tanto al ejecutar la aplicación como en la función @Preview.
- Al combinar el filtro por nombre con el menú desplegable de categorías, y agregar íconos y colores distintos para cada una, nos damos cuenta que una búsqueda es realmente útil cuando permite varios criterios y muestra el resultado con claridad.

También comprobé que hay que manejar de forma explícita los casos límite, como cuando una búsqueda no tiene coincidencias.

Recomendaciones

- Es mejor mantener el texto de búsqueda y la categoría seleccionada en el Composable contenedor y pasar a los componentes hijos solo los valores que necesitan. Así FilaEdificio o FiltroCategoria no manejan estados propios que podrían desincronizarse, y resulta más fácil añadir nuevos filtros sin rehacer la estructura visual.
- Envolver el cálculo de la lista filtrada en remember(texto, categoriaSeleccionada, edificios) { ... }, de modo que solo se ejecute cuando alguno de esos valores cambie y no en cada recomposición. Es una práctica sencilla que ayuda a distinguir entre cálculos costosos y trabajo trivial, y que sirve de base para pasar luego a derivedStateOf y ViewModel en proyectos más grandes.
- Recomiendo reforzar el manejo de datos inválidos en parsearEdificios, validando identificadores duplicados, etc. También sería bueno definir cómo actualizar el archivo de filesDir cuando cambie la versión incluida en assets, en vez de depender de borrar los datos de la aplicación.
- Por último, sugiero agregar desde el principio pruebas unitarias para parsearEdificios y filtrarEdificios, ya que son funciones puras que se pueden verificar sin emulador, y complementarlas con pruebas de interfaz para los casos de lista vacía, coincidencia exacta y combinación de filtros.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] I. Sommerville, *Ingeniería del Software*, 9.ª ed. Madrid: Pearson Educación, 2011.

[2] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7.ª ed. México: McGraw-Hill, 2010.

[3] Escuela Profesional de Ingeniería de Sistemas, «Guía de Prácticas de Laboratorio». 2024.